

```

def search(self):
    while self.open_nodes != []:
        node = self.open_nodes.pop(0)
        if self.problem.goal_test(node.state):
            self.solution = node
            return self.get_path(node)
        lnewnodes = []
        for a in self.problem.domain.actions(node.state):
            newstate = self.problem.domain.result(node.state,a)
            if newstate not in self.get_path(node):
                newnode = SearchNode(newstate,node)
                lnewnodes.append(newnode)
        self.add_to_open(lnewnodes)
    return None

```

$\neq$ node caminho
 if newstate not in self.get_path(node):
 newnode = SearchNode(newstate,node)
 lnewnodes.append(newnode)

Se for verdade quer dizer que o nó
já está repetido e evita loops infinitos

```

def __init__(self, state, parent):
    self.state = state
    self.parent = parent
    self.depth = 0 if parent == None else parent.depth + 1

```

Se ele for a raiz é igual
a 0, caso contrário é a
profundidade do pai

```

if self.problem.goal_test(node.state):
    self.solution = node
    self.lenght = node.depth

```

comprimento

Até ao número 6

5. Acrescente código ao método `search()` da classe `SearchTree` por forma a calcular o número total de nós terminais (folhas) e não terminais (nós já expandidos, mesmo que sem filhos) existentes na árvore após a conclusão da pesquisa. Essa informação deverá ficar armazenada em atributos do `self`.

1. Criar novos atributos (c/terminais & s/terminais)

→ `self.terminal` e `self.nonterminal`

Casos:

- Cada nó expandido → `nonterminal += 1`
- Se não for expandido → `terminal += 1`
- Fim da pesquisa → `terminal = len (quem - nodes)`
(somados)

6. O factor de ramificação média é dado pelo ratio entre o número de nós filhos (ou seja, todos os nós com excepção da raiz da árvore) e o número de nós pais (nós expandidos ou não terminais). Inclua na árvore de pesquisa uma @property `avg_branching` que devolva o respectivo factor de ramificação média.

Ramificação Média - o valor da média de quantos
filhos tem cada nó pai numa tree search

Nota

- Preciso de criar um novo atributo → `self.avg`

```
@property
def avg_branching(self):
    # O número de nós filhos é o total de nós gerados menos 1 (a raiz)
    # O número de nós pais é o número de nós expandidos (non_terminals)
    if self.non_terminals == 0:
        return 0
    # Para garantir que o cálculo é feito após a busca
    total_nodos = self.non_terminals + self.terminals
    filhos = total_nodos - 1
    pais = self.non_terminals
    return filhos / pais if pais > 0 else 0
```

💡 Todos os nós **exceto** a raiz são **filhos** de algum pai

Logo se tiver **N** nós no final de uma busca, o número de nós que são filhos são todos menos o primeiro (raiz) **N - 1**.

💡 Os **nós pais** dão os nós que geraram pelo menos 1 filho, logo nós são terminais (**self.non_terminal**)

